

We Can Use *Macros* In Rust For Print Values, Every Thing In Rust That Ends With `!` Is *Macro*:

```
fn main() {  
    let y = 10;  
  
    // y = 20; // This Will Raise Compile Error  
  
    let mut x = 20;  
  
    println!("The Value Of X, Before Changing Is: {}", x);  
  
    x = 25; // If We Don't Use The Previous Value, Warning Will Appear  
  
    println!("Hello, world!, y is: {}, x is: {}, x+y is: {}", y, x, y+x);  
}
```

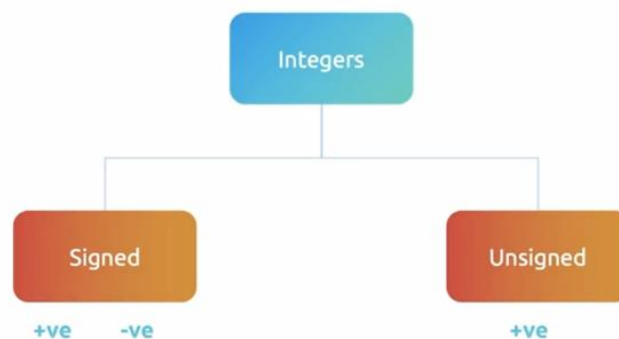
Rust Is Static Typing Language, That Means It Must Know The Type Of Variables In Compile Time

Data Types In Rust Can Be Combine In Two Groups: *Scaler And Compound Data Types*

Scaler Data Types



Integers



Rust Integer Types

	Length	Length	Length
	8-bit	i8	u8
	16-bit	i16	u16
Default	32-bit	i32	u32
	64-bit	i64	u64
	128-bit	i128	u128
	arch	isize	usize

Floating Integers



```
/*  
    * Testing The Integer Types  
    */  
let a1 = 98_999;  
let a2 = 0xffff;  
let a3 = 0o337;  
let a4 = 0b1111_0000;  
let a5 = b'a';  
  
println!("a1 is: {}, a2 is: {}, a3 is: {}, a4 is: {}, a5 is: {}", a1, a2, a3,  
a4, a5);  
*****  
/*  
    * Testing Floating Types  
    */  
let a6 = 2.5;  
let a7: f32 = 4.9;  
println!("a6 is: {}, a7 is: {}", a6, a7);  
*****
```

Boolean

Primarily used in conditional statements

```
main.rs
fn main() {
    let t = true;
    let f: bool = false; //with explicit type annotation
}
```

Represents a Unicode Scalar Value (ASCII characters)

```
main.rs
fn main() {
    let c = 'z';
    let z: char = 'Z'; //with explicit type annotation
    let heart_eyed_cat = '😺';
}
```

Can compound [data](#) at once.

It has primary types: *Tuples* and *Arrays*.

Compound Data Types

Tuples

Arrays

```

let tup: (i32, f64, u8) = (500, 6.5, 1);

// We can destructing tuple:
// Here we can assign types for each variable
let (x, y, z) = tup;

println!("The value of x is: {}, the value of y is: {}, the value of z is: {}",
x, y, z)

// Testing tuples
let tup: (i32, f64, u8) = (500, 6.5, 1);

// We can destructing tuple:
let (x: i32, y: f64, z: u8) = tup;
You, 1 minute ago • Uncommitted changes
println!("The value of x is: {}, the value of y is: {}, the value of z is: {}", x, y, z)

*****

// Access the tuple element directly, using . then index
// also we can set the type of each variable
// index start from 0
let x1 = tup.0;
let y1: f64 = tup.1;
let z1 = tup.2;

println!("The value of x1 is: {}, the value of y1 is: {}, the value of z1 is:
{}", x1, y1, z1);
*****

```



```

fn main() {
    let a = [1, 2, 3, 4, 5];

    let first = a[0];
    let second = a[1];
}

```

Arrays are collections of values of the same type. They have a fixed length.

```

*****

```

```
// Testing Arrays
let arr = [1, 2, 3, 4, 5];

// println!("The Array is: {}", arr); // This will give compiler error
let temp = arr[3];

println!("The 1st value is: {}, the 2nd value is: {}, the 3rd value is: {}, the
4th value: {temp}", [0], arr[1], arr[2])
*****

// Testing Arrays
let arr: [i32; 5] = [1, 2, 3, 4, 5];

// println!("The Array is: {}", arr); // This will give compiler error

let temp: i32 = arr[3];

println!("The 1st value is: {}, the 2nd value is: {}, the 3rd value is: {}, the 4th value: {temp}",
arr[0], arr[1], arr[2])
*****
```

Rust checks at runtime to ensure the index is within bounds. If it's not, Rust will panic to prevent accessing invalid memory.

```
*****
// for better memory safety, when dealing with arrays and indexes
let index: usize = 3;

match arr.get(index) {
    Some(element: &i32) => println!("The element is: {element}"),
    None => println!("Please check the index"),
}
```

To handle invalid array element access gracefully, we can use methods like "get."

```
*****
```

01 Constants are values that are bound to a name and are not allowed to change.

02 They are always immutable and are declared using the keyword `const`.

```
main.rs
const MAX_USERS: u32 = 1000;
```

Constant with a value of 1,000

Constant with unsigned 32-bit integer

Note 1: The constants in rust must be all uppercase with `_` between words.

Note 2: We must declare the type of constant.

```
// Testing Constants
const MAX_USERS: u32 = 1000;
```

```
println!("The max users is: {MAX_USERS}");
```

The 1 means: the global scope.

The 03 means: we can set it to function the generate different values.

Differences Between Constants and Variables

01 Constants are always immutable. We can't use the `mut` keyword with constants.

02 Constants can be declared in any scope.

```
main.rs
const SERVER_ADDRESS: &str = "192.168.1.1";

fn main() {
    println!("Server address: {}", SERVER_ADDRESS);
}
```

03 Constants may only be set to a constant expression.

```
main.rs
const SECONDS_IN_A_MINUTE: u32 = 60;
const MINUTES_IN_AN_HOUR: u32 = 60;
const HOURS_IN_A_DAY: u32 = 24;
const SECONDS_IN_A_DAY: u32 = SECONDS_IN_A_MINUTE *
    MINUTES_IN_AN_HOUR * HOURS_IN_A_DAY;
```

Shadowing in rust means: The variable can be used multiple times.

```
// Testing Shadowing
```

```
let a = 5;
```

```
let a = a + 1; // Shadowing the first 'a'
```

```
println!("The value of a is: {a}")
```

```
{
```

```
    let a = a * 2; // Shadowing within new scope
```

```
    println!("The value of a in inner scope: {a}");
```

```
}
```

```
println!("The value of a outside scope is: {a}");
```

Why Use Shadowing?

```
main.rs

fn main() {
    let spaces = "  "; // `spaces` is a string
    let spaces = spaces.len(); // Now, `spaces` is
a number
    println!("The length of spaces is: {}",
spaces); // Outputs: 3
}
```

It allows us to reuse variable names without mutability.

It provides flexibility to change variable types and helps keep your code concise and readable.

The variable "spaces" first holds a string and then its length as an integer. This flexibility simplifies the code and helps avoid the need for additional variable names.

```
let spaces: &'static str = "  ";
let spaces: usize = spaces.len();
```

You, 2

```
println!("The spaces value is: {spaces}");
```

Macros

Macros in Rust are a way of writing code that writes other code, which is known as **metaprogramming**.

01

Declarative
Macros

02

Procedural
Macros

```
*****
use std::io;

fn main() {
    println!("Please enter your age: ");

    let mut input = String::new();

    io::stdin().read_line(&mut input).expect("Failed to read line");

    let input = input.trim();

    let input: u32 = input.parse().expect("Please enter a number!");

    println!("Your age is: {input}");
}
*****
```

Declarative Macros



```
*****
```

To pass parameters to macro:

Note: here expr means that name can be any valid rust expr (variables)

```
macro_rules! greet {  
    ($name:expr) => { // Here also we can pass no arguments  
        println!("Hello {}", $name)  
    }  
}
```

Practical Use Cases

01 Logging Macros

02 println! Macro



```
main.rs  
  
macro_rules! log {  
    ($msg:expr) => {  
        println!("[LOG]: {}", $msg);  
    };  
}  
  
fn main() {  
    log!("Application started");  
}
```

Control flow

Control flow is a fundamental concept in programming that allows you to dictate the execution path of your code based on certain conditions.

Positive

Negative

Zero

In Rust, control flow is managed using conditional statements like if, else if, and **else**

```
fn main() {  
    let number: i32 = 7;  
    if number > 10 {  
        println!("ERROR!!!");  
    } else if number > 5 {  
        println!("TRUE!!!");  
    } else {  
        println!("ERROR!!!");  
    }  
}  
*****
```